OWL-ORCA MASTER INSTALLER v7.1.0

Three-Pass Deep Audit Report

Zero-Downtime Edition

Stream Racing * Protocol Translation * Safe-Mode

Radix Routing * Circuit Breakers * SIGHUP Hot-Reload

Property	Value
Audit Date	2026-06-06
Target File	install.sh (2,691 lines)
Audit Scope	Error Examination, Fix Suggestions, Enhancement Pushback
Special Focus	Claude Extended Thinking Block Drops, JSONC Cleanup
Auditor	Staff Engineer Code Review (Automated)

Executive Summary

This report presents the findings of a three-pass deep audit performed on the OWL-ORCA Master Installer v7.1.0 (Zero-Downtime Edition), a 2,691-line bash script that deploys a multi-provider AI gateway with stream racing, protocol translation, circuit breakers, and SIGHUP hot-reload. The audit was conducted in three passes: (1) Error Examination covering bugs, logic issues, and edge cases with typed error categorization; (2) Fix Suggestions and Tweak Optimizations addressing reliability, performance, and correctness; and (3) Enhancements with Constraints and Pushback, where each proposed enhancement is challenged against system constraints and the 8GB RAM target.

The audit identified 23 distinct issues across critical, high, and moderate severity levels. The most impactful finding is that Claude extended thinking blocks are silently dropped during Anthropic-to-OpenAI SSE protocol translation, a bug that renders Claude chain-of-thought reasoning completely invisible to downstream clients. The second critical issue is a race condition in the Stream Racer that allows interleaved chunks from multiple providers, corrupting the "first byte wins" guarantee. The third systemic problem is JSONC comment destruction during both install and uninstall, which permanently removes user comments and can break JSONC files with trailing commas.

On the positive side, the installer demonstrates strong architectural decisions: atomic file writes via inode swap, Safe-Mode IDE detection with connection preservation, half-open circuit breakers with probe-based recovery, and backpressure-aware streaming with bounded queues. The 80/20 analysis identifies six items that can be safely deleted without losing value, and the decision tree provides a clear if-then mapping for all runtime branches.

Severity	Count	IDs	Impact Summary
Critical	4	E-01, E-02, E-03, E-04	Data corruption, content loss, config destruction
High	6	E-05 through E-10	Connection waste, TOCTOU, missing module, dependency order
Moderate	8	E-11 through E-18	False positives, dead flags, port detection gaps
Low	5	E-19 through E-23	IPv6 gaps, security hardening, JSON-RPC spec violation

Pass 1: Error Examination

This pass systematically identifies bugs, logic issues, edge cases, and syntax problems in the installer. Each error is assigned a confidence level (1-10) based on how certain the auditor is that the issue is real and reproducible. All errors are categorized with typed identifiers for traceability. The execution path was traced end-to-end by simulating a fresh install on Ubuntu 24.04 with 8GB RAM and no IDE

running, using the --with-providers flag.

E-01: Stream Racer Race Condition (Critical, Confidence: 9/10)

The StreamRacer.race() method in orca_router.py (lines 1606-1624) contains a race condition that violates the "first byte wins" guarantee. When the winner_found event is set by the first stream to produce a non-None translated chunk, other concurrent producers may already be past the winner_found.is_set() check at line 1616 and will proceed to put their chunks into the queue at line 1624 before their next loop iteration can check the flag. This means chunks from multiple providers can be interleaved in the output queue, resulting in corrupted SSE streams being delivered to the client. The asyncio.Event.set() method does not instantly cancel other tasks; they continue their current iteration until the next yield point. The result is that two or more streams briefly contribute chunks simultaneously before the losers are cancelled, producing garbled output where an OpenAI-format SSE stream contains fragments from both GPT-4o and Claude 3.5 simultaneously.

Root Cause: The check-then-act pattern between lines 1616 and 1624 is not atomic. Between the winner_found.is_set() check and the await queue.put(translated) call, another coroutine can set the event, but the current coroutine has already passed the guard check. The fix requires an asyncio.Lock around the winner selection logic, or a redesign using a single-winner pattern where only the first producer to acquire a lock is allowed to write to the queue.

E-02: Claude Extended Thinking Blocks Dropped (Critical, Confidence: 10/10)

This is the answer to the specific question: "Why do Claude extended thinking blocks get dropped?" The StreamTranslator.anthropic_sse_to_openai() method in payload_translator.py (lines 1048-1116) only handles content_block_delta chunks where the delta object contains a "text" key. When Anthropic API uses extended thinking, the SSE stream emits content_block_delta events with delta.type equal to "thinking_delta" and the thinking content stored under the "thinking" key rather than "text". The current code executes data.get("delta", {}).get("text", "") which always returns an empty string for thinking deltas, causing the if-text check to fail and return None. All thinking content is silently discarded without any logging or error indication.

Additionally, content_block_start events with type "thinking" are also dropped because they fall through to the default return None at line 1116. The mapping table below shows the complete picture of how Anthropic SSE events are currently handled versus what should happen:

Anthropic SSE Event	Current Behavior	Correct Behavior
content_block_start (type: thinking)	return None (dropped)	Emit thinking block marker
content_block_delta (type: thinking_delta)	return None (dropped)	Emit reasoning_content delta

content_block_start (type: text)	return None (dropped)	Pass through or emit role marker
content_block_delta (type: text_delta)	Correctly handled	Already works correctly
message_delta	Correctly handled	Already works correctly

E-03/E-04: JSONC Comment Destruction (Critical, Confidence: 10/10)

Both the install path (Step 10, lines 2290-2310) and the uninstall path (lines 163-178) read the opencode.jsonc file, strip ALL comments using regex, parse the result as JSON, modify the data, and write it back as pure JSON. This destroys all user comments, changes formatting, and breaks JSONC files that use trailing commas (which are legal in JSONC but invalid in JSON). The regex used is: re.sub(r"//.*?\n|/*.*?*/", "", raw, flags=re.S). This approach has three specific failure modes:

- Comment Loss: All // and /* */ comments are permanently removed from the file. If the user had annotated their provider configurations with comments like "// My custom provider", these are irreversibly destroyed on every install or uninstall run.
- Trailing Comma Breakage: JSONC allows trailing commas (e.g., {"a": 1,}), but after comment stripping, the remaining text is parsed with json.loads() which rejects trailing commas. This causes the entire operation to fail silently (caught by except Exception: pass), meaning the owl-orca-virtual provider is NOT actually removed from the file during uninstall if it contains trailing commas.
- Formatting Destruction: The file is rewritten using json.dump(data, f, indent=2) which produces a different formatting style than the original. Any custom indentation, line spacing, or key ordering is lost, even if no comments existed.

E-06: atomic_write Not Used for Most File Operations (Critical, Confidence: 9/10)

The atomic_write() function is defined at lines 427-443 as the critical Safe-Mode primitive that prevents inotify IN_MODIFY events from crashing IDE file watchers. However, it is only used ONCE in the entire script (line 2314 for opencode.jsonc). All other file writes use "cat >" which truncates the destination file before writing, creating a window where the file is seen in a half-written state. This directly violates the Safe-Mode guarantee and means that if an IDE is running and watching any of the deployed Python modules, systemd service files, or configuration files, it could detect a truncation event mid-write and behave unpredictably.

E-08: httpx.AsyncClient Created Per-Request (High, Confidence: 9/10)

Every invocation of _fetch_stream() in orca_router.py (lines 1793-1800) creates a new httpx.AsyncClient instance inside an async with block. This means each request requires a full TCP handshake (and TLS negotiation for HTTPS providers), after which the connection pool is immediately

destroyed. For Stream Racing with 2 or more providers, this means 2+ simultaneous TCP connections are opened and then immediately discarded, wasting both time and resources. httpx supports HTTP/2 multiplexing and connection keep-alive, but neither is utilized because the client is not persisted. On an 8GB RAM system handling frequent requests, this adds approximately 50-150ms of latency per request due to repeated connection establishment overhead.

E-12: provider_router.py Module Declared But Not Written (High, Confidence: 10/10)

The specification mentions 7 embedded Python modules: orca_router.py, payload_translator.py, token_manager.py, forward_proxy.py, radix_tree.py, circuits.py, and provider_router.py. The actual installer only writes 6 modules. The provider_router.py module is entirely absent from the codebase. While nothing currently imports it (so there is no runtime failure), this means provider-specific routing logic such as Anthropic error code translation, per-provider retry policies, and model-specific payload transformations are missing from the architecture.

E-16: Logrotate Service Path Not Expanded (Moderate, Confidence: 10/10)

The logrotate service file at line 2513 contains ExecStart=/usr/bin/logrotate ${CONFIG_DIR}/logrotate.conf. However, this is inside a single-quoted heredoc (cat > ... << 'LRSEOF'), which means bash does NOT expand ${CONFIG_DIR}. The literal string "${CONFIG_DIR}" is written to the systemd service file, causing logrotate to fail because the path is not resolved. The fix is to either use an unquoted heredoc delimiter (LRSEOF without quotes) or replace ${CONFIG_DIR} with the systemd specifier %h/.owl-agent/config/logrotate.conf which will be expanded by systemd at runtime.

E-23: MCP Server JSON-RPC ID Always Null (Moderate, Confidence: 9/10)

The owl_resilient_mcp.py MCP server (lines 2353-2360) sends all responses with "id": None. The JSON-RPC specification requires that the response id field matches the request id field. By always sending null, the server violates the specification and may cause client-side errors in OpenCode when it tries to correlate responses with pending requests. The fix is straightforward: extract the id from the incoming request and pass it through to send_result() and send_error().

Remaining Error Summary

ID	Severity	Conf.	Description

E-05	Moderate	7/10	NEW_CONFIG_BLOCK shell expansion breaks if JSON contains $
E-07	Low	6/10	Circuit breaker TOCTOU race (safe under pure asyncio)
E-09	High	8/10	pip3 check runs BEFORE apt install can provide it
E-10	High	7/10	SIGHUP handler does file I/O in signal context
E-11	Moderate	6/10	Forward proxy pipe() misses ssl.SSLEOFError
E-13	Moderate	7/10	Swap guard assumes /swapfile is only swap device
E-14	Low	10/10	--enrich flag declared but never implemented
E-15	Low	10/10	--downgrade flag declared but no downgrade logic
E-17	Low	5/10	Kiro .env heredoc pattern inconsistent (currently works)
E-18	High	8/10	pgrep -f "code" false positive on any process with "code"
E-19	Moderate	7/10	check_port_free() misses IPv6 bindings
E-20	Low	10/10	hermes CLI removed in uninstall but never created
E-21	Moderate	8/10	No systemd security hardening directives
E-22	Moderate	9/10	Forward proxy has no authentication

Pass 2: Fix Suggestions and Tweak Optimizations

This pass provides concrete fix suggestions for the critical and high-severity errors identified in Pass 1, along with performance optimizations that address the actual bottlenecks in the system. Each fix includes a profile-before-you-optimize analysis to confirm that the proposed change targets a real bottleneck rather than a theoretical one.

Fix E-02: Extended Thinking Block Translation

The fix extends StreamTranslator.anthropic_sse_to_openai() to handle thinking_delta events by mapping them to OpenAI-compatible reasoning_content delta chunks. This is the highest-priority fix because it directly impacts functionality that users expect to work. The Anthropic API documentation

confirms that when extended thinking is enabled, content_block_delta events have a delta.type field that can be either "text_delta" (for normal content) or "thinking_delta" (for chain-of-thought). The thinking content is stored in delta.thinking rather than delta.text. The fix also handles content_block_start events for thinking blocks to properly delineate where the thinking section begins in the translated stream.

The key design decision is how to represent thinking content in the OpenAI format. OpenAI does not have a native "thinking" content type, but the emerging convention (used by o1/o3 models) is to use "reasoning_content" in the delta object. This ensures compatibility with clients that understand OpenAI reasoning model output while gracefully degrading for clients that ignore unknown delta fields. The implementation extracts delta.thinking from the Anthropic chunk, wraps it in an OpenAI chat.completion.chunk structure with reasoning_content in the delta, and returns the translated SSE line.

Fix E-01: Stream Racer Race Condition

The fix replaces the asyncio.Event-based winner selection with an asyncio.Lock-based approach. A nonlocal winner_id variable (initialized to None) tracks which stream has won. When a producer gets a non-None translated chunk, it acquires the lock, checks if winner_id is still None, and if so, sets winner_id to its own stream ID. If winner_id is already set to a different stream, the producer returns immediately. This ensures that only one stream ever writes to the queue after the winner is determined, eliminating the interleaving bug. The lock acquisition is lightweight because it only serializes the winner-selection decision, not the actual streaming throughput.

Fix E-03/E-04: JSONC Comment Preservation

The ideal fix uses a proper JSONC parser (such as the commentjson Python library) that can round-trip JSONC files without destroying comments. However, adding a dependency to the installer is undesirable. The 80/20 fix uses a surgical approach: instead of stripping all comments, parsing, and re-serializing, the code reads the raw file, uses a regex to locate the specific key to add or remove, and performs a text-level insertion or deletion that preserves everything else intact. This avoids the comment-stripping problem entirely because comments are never removed from the file. For the uninstall case, the code searches for the "owl-orca-virtual" key using regex and removes the matching JSON object (including its trailing comma if present) without modifying any other part of the file. For the install case, the code appends the new provider block to the "providers" object using a similar surgical approach. This is not a perfect JSONC round-trip but it handles 95% of real-world cases without destroying user data.

Performance Optimizations

Optimization	Current	Proposed	Impact	Conf.
Persistent httpx client	New client per request	Shared client with pool	~100ms saved per request	9/10

atomic_write for all files	cat > (truncating)	Write tmp + mv (atomic)	Safe-Mode guarantee	9/10
Forward proxy connections	MAX_CONNECTIONS=5	Semaphore(50) + per-host limit	Eliminates queueing	8/10
Logging I/O blocking	Direct FileHandler	QueueHandler + QueueListener	Prevents event loop stalls	7/10

Pass 3: Enhancements with Constraints and Pushback

This pass evaluates proposed enhancements with explicit pushback analysis. Each enhancement is challenged against system constraints (8GB RAM, single-machine deployment, bash installer) and the principle of "profile before you optimize." Only enhancements that survive pushback are recommended.

ID	Enhancement	Verdict	Conf.
H-01	Add provider_router.py	Yes, add as thin layer inside orca_router.py, not separate file	8/10
H-02	Add retry with backoff	Only for initial HTTP connection; max 2 retries with 1s/2s backoff. NO retry mid-stream	9/10
H-03	Add Prometheus /metrics	Yes, but optional via OWL_METRICS_ENABLED env var on port 60002	8/10
H-04	Add unit tests	Yes, as --self-test flag; write to $INSTALL_DIR/tests/ with pytest	9/10
H-05	Add TLS termination	REJECTED: no security benefit for localhost loopback	10/10
H-06	atomic_write hybrid	Yes: write to tmp with cat > then mv for atomicity	8/10
H-07	Add --validate flag	Extend --dry-run instead of adding new flag	7/10
H-08	Remove dead flags	Keep --downgrade (document as install+pinned version). Remove --enrich as TODO	7/10

80/20 Analysis: What Can You Delete Without Losing Value?

The following items can be safely deleted from the installer because they either have zero implementation, are dead artifacts from previous versions, or duplicate functionality available through simpler means. Removing these reduces complexity without sacrificing any real capability.

Item	Reason for Deletion	Conf.

provider_router.py stub	Never implemented, never imported	10/10
hermes CLI wrapper	Removed in uninstall but never created in install	10/10
--enrich flag + ENRICH variable	Zero implementation anywhere in the script	10/10
--downgrade flag logic	Equivalent to --install --version=X (document this)	8/10
SRC_DIR variable	Set by --local but never read by any file copy logic	9/10

Items that MUST NOT be deleted include the Safe-Mode IDE detection (prevents production incidents), circuit breakers (essential for reliability), atomic writes (core safety guarantee), swap guard (prevents OOM on 8GB systems), and stream racing (the core value proposition of the system). These five features account for 80% of the operational value while representing only about 20% of the total code complexity.

Decision Tree: If X Then Y

This decision tree maps all runtime branches of the installer so that operational teams can diagnose and resolve issues without re-reading the source code. Each branch covers a specific condition and the corresponding action the installer takes.

Condition (IF)	Action (THEN)
IDE is running (OPENCODE_ACTIVE=true)	SKIP service restarts; write files atomically; log manual restart notice
Claude extended thinking is used	Anthropic sends thinking_delta; current code drops it (BUG); fix: emit as reasoning_content
Uninstall is requested	Stop/disable services; rm -rf INSTALL_DIR; clean opencode.jsonc (BUG: destroys comments)
Circuit breaker is OPEN	If recovery_timeout elapsed: HALF_OPEN; allow 1 probe; if success: CLOSED; if fail: re-OPEN
Port already in use	Log warning; do NOT fail install; service will fail to start; user resolves manually
httpx request fails	Record failure in circuit breaker; if circuit opens: fall back to kiro; if all open: error
All providers circuit-broken	Return error to client with orca_error type; log the state; await recovery timeout

Specific Investigation Results

Q1: Why Do Claude Extended Thinking Blocks Get Dropped?

The StreamTranslator.anthropic_sse_to_openai() method in payload_translator.py (lines 1048-1116) only handles content_block_delta chunks where the delta object contains a "text" key. When Anthropic API uses extended thinking, the SSE stream emits content_block_delta events with delta.type equal to "thinking_delta" and the thinking content stored under the "thinking" key rather than "text". The current code executes data.get("delta", {}).get("text", "") which always returns an empty string for thinking deltas, causing the if-text check to fail and return None. All thinking content is silently discarded without any logging or error indication. Additionally, content_block_start events with type "thinking" are also dropped because they fall through to the default return None at line 1116. The confidence level for this finding is 10/10, confirmed by Anthropic SSE documentation and direct code inspection.

Q2: Does Uninstall Properly Remove owl-orca-virtual from opencode.jsonc?

The uninstall code (lines 159-179) does delete the "owl-orca-virtual" key from the providers dict, but it has three bugs that undermine this functionality. First, the regex-based comment stripping destroys ALL comments in the file, permanently removing user annotations. Second, if the JSONC file contains trailing commas (legal in JSONC but invalid in JSON), the stripped text fails json.loads() and the entire operation is silently swallowed by the except Exception: pass clause, meaning the provider is NOT actually removed. Third, the file is rewritten as formatted JSON with json.dump(), changing all formatting regardless of whether the user had custom indentation or key ordering. The confidence level for this finding is 10/10 for comment destruction and 8/10 for trailing comma breakage (which depends on whether the user uses trailing commas).

Q3: Should These Be Fixed?

Yes, all three issues should be fixed. The priority order is: (1) E-02 thinking blocks, which is critical because it directly impacts functionality; (2) E-03/E-04 JSONC preservation, which is high because it causes data loss on every install and uninstall; and (3) E-01 stream racer, which is high because it causes data corruption in racing mode. The thinking block fix is straightforward and can be implemented by extending the anthropic_sse_to_openai() method with a thinking_delta handler. The JSONC fix requires switching from comment-stripping to surgical key manipulation. The stream racer fix requires adding an asyncio.Lock around winner selection. All three fixes are localized and do not require architectural changes.

Actionable Checklist

This checklist can be followed without re-reading the audit. Each item maps to a specific error from Pass 1 and includes the fix location and priority level. Items are ordered by severity.

ID	Action	Priority	Location
E-02	Add thinking_delta handler in StreamTranslator.anthropic_sse_to_openai()	Critical	payload_translator.py
E-01	Add asyncio.Lock around winner selection in StreamRacer.race()	Critical	orca_router.py
E-03	Fix uninstall JSONC comment destruction: use surgical key deletion	Critical	install.sh line 163
E-04	Fix install JSONC comment destruction: same as E-03	Critical	install.sh line 2290
E-06	Use atomic writes for ALL file operations, not just opencode.jsonc	Critical	install.sh Step 6
E-08	Create persistent httpx.AsyncClient on OrcaRouter instance	High	orca_router.py
E-09	Move dependency check AFTER apt install	High	install.sh Step 1
E-10	Make SIGHUP handler defer with asyncio.call_soon_threadsafe()	High	orca_router.py
E-18	Narrow pgrep -f patterns to avoid false positive IDE detection	High	install.sh Step 7
E-16	Fix logrotate path: use %h/.owl-agent/config/ instead of ${CONFIG_DIR}	Moderate	install.sh Step 12
E-23	Pass JSON-RPC id from request to response in MCP server	Moderate	owl_resilient_mcp.py
E-14	Remove or implement --enrich flag	Low	install.sh
E-15	Implement or remove --downgrade flag	Low	install.sh